

ASEN 5264 Final Project : BSK-RL Based Reinforcement Learning for Orbit Attainment and Collision Avoidance

Gabriel Blandin

Brian Lumsden

Cannon Palmer

Abstract—This paper illustrates basic reinforcement learning (RL) applications at 3 levels relative to common spacecraft tasks. The first involves a spacecraft maneuvering to find randomized optimal low-Earth orbits in which to park after starting in some randomized orbit itself. The second involves a spacecraft minimizing fuel utilization and maximizing time spent in its prescribed orbit while avoiding collision with resident space objects (RSOs) in its path. The third builds upon the second by utilizing a different RL algorithm, adding extra reward criteria, and introducing further complexity to the action and observation spaces.

Index Terms—spacecraft tasking, reinforcement learning, autonomy

I. INTRODUCTION

As research and exploration in space continue and expand, more spacecraft and debris will occupy the space around Earth. With this buildup, collisions between space objects becomes more likely, and improvements in collision avoidance technologies are necessary. One particularly important advancement should be a substantial decrease in dependence on human intervention, as relying on that will become unsustainable. A solution to this problem is to use artificially intelligent autonomous systems onboard spacecraft to more efficiently handle critical collision avoidance decision making [1]. With proper training, these autonomous systems can learn to take actions to change their host spacecrafts' trajectories based on what they know about their surroundings, all without manual human intervention. This capability proves valuable for the aforementioned collision avoidance problem and can be a solution for other problems like rendezvous with another spacecraft or, perhaps, orbital adjustments after conducting collision avoidance maneuvers. With trained autonomous systems managing these processes, organizations that conduct research and exploration in space, with increasing volume and frequency, are likely to operate in a safer and more sustainable environment [1].

Our initial proposal for this project was perhaps slightly over ambitious. While it was within the scope of the course, BSK-RL itself was not immediately usable with simplistic approaches like forward search. This led to us utilizing PPO for our minimum approach. This is because of the framework's structuring for actions, state space etc. We later found some potential ways to better discretize the spaces, but these were not known to us when first beginning and working through our minimum approach. For our main approach we also changed

our approach from our proposal by choosing to use DQN instead of SARSA- λ based on the research conducted in [2]. Had we begun our work with a firmer understanding of the framework itself, we may have been better able to structure our initial proposal and better understood what tools would have likely served each approach best.

II. RELATED WORK

[1] served as valuable background for existing research into similar topics. However, this paper did not seem to use BSK-RL. Regardless, it gave us insight into approaches for problem formulation and modeling.

[2] provided more background for similar problems where satellite obstacle avoidance of a single Resident Space Object (RSO) is performed in a Low Earth Orbit (LEO), but it did not use BSK-RL. It discussed implementation ideas for multiple RL algorithms that were useful to us, and it highlighted the effectiveness of PPO in complex and continuous problem spaces. Specifically, it provided ideas for discretizing the continuous action space for use with the DQN algorithm and methods to setup an effective PPO algorithm within the continuous action space.

[3] is the paper that introduced the BSK-RL framework and made this paper possible. This paper outlines the lack of open source RL environments for the space domain. The authors sought to fill this gap by creating a bridge between the Basalisk simulation package and the Gymnasium RL framework. The paper served as a good reference for the intent of BSK-RL and initial working examples.

[4] discussed obstacle avoidance with multiple RSOs and demonstrated the formulation of the problem into a CMDP (an MDP with an additional space for constraints), which proved to be a valuable reference as we defined our own base MDP problem and expanded to a POMDP. Two reward functions were described here for reducing energy consumption and reducing collision probability which influenced our reward function design.

[5] expanded our knowledge of collision avoidance in a LEO orbit, including optimizing delta V maneuvering to achieve different collision avoidance goals.

III. POMDP FORMULATION

Our problem space was defined in BSK-RL using a POMDP structure: $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \mathcal{O}, \mathcal{Z}, \gamma)$

A. State Space $\mathcal{S}$

The state space in BSK-RL is described as the combination of all states necessary for all components in the simulation and environment. Some examples include: the orbital elements of each satellites' orbit, the remaining fuel each satellite has, relative positions and velocities between satellites, etc. The state space in this problem is too large for direct learning [2].

B. Minimum Action Space $\mathcal{A}$

The action space is defined by the combination of all available actions that can be taken by each satellite in the environment. In our minimum goal, the action space consists of only our lone satellite's actions. The action this minimum satellite can take is called ImpulsiveThrust in BSK-RL. When executing this ImpulsiveThrust action, the satellite can change its velocity by the specified dv values in all 3 Hill Frame axis directions, and then drift for a specified duration. This action space is continuous, and we have enforced a maximum combined dv magnitude of 100.0 and a maximum drift time of 5700.0

$$\mathcal{A} = \left\{ (dV, \text{drift}) \in \mathbb{R}^3 \times \mathbb{R} \mid \|dV\| \leq 100.0, \right. \\ \left. 2.0 \leq \text{drift} \leq 5700.0 \right\} \quad (1)$$

C. Main Action Space $\mathcal{A}$

In our main goal of obstacle avoidance, the action space technically becomes larger. In addition to the ImpulsiveThrust action of the main satellite, each RSO must be given an action according to BSK-RL. We define the RSOs to have a constant action of pointing Nadir, which means to point directly towards the center of the Earth. Thus, the action space present in the environment has expanded to include the point Nadir action for each RSO. Fortunately, the RSOs have a trivial action space since they only have the one action of pointing Nadir; therefore our RL models can ignore training on the RSO actions and simply focus on training within the main satellites ImpulsiveThrust action space.

D. Transition Function $\mathcal{T}$

The transition function is deterministic according to the generative model provided by the environment:

$$\mathcal{T}(G(s, a) | s, a) = 1 \quad (2)$$

E. Minimum Reward $\mathcal{R}$

We defined the reward function according to each of our goals. In our minimum goal, our reward function is represented by a MinimumReward class that defines rewards earned by the agent based on its current orbit. The reward value is determined from the proximity of the agent's orbital parameters to the orbital parameters of a reward orbit which is initially randomized by the environment within a set of orbital parameter bounds we have set. The reward orbit, referred to as *reward orbit* in our code, is described by a semi major axis (m), an eccentricity, an inclination (rad), and an associated reward randomized between 1 and 100 by default. The agent receives the reward associated with the reward orbit if its orbital

parameters are within a range of acceptable tolerances that correspond to each parameter. The tolerance values, defined in their own class called MinimumRewardTolerances, account for the continuous state space since it is near impossible to reach the exact orbit. We also define a partial credit function that assigns negative rewards to all orbits the satellite can have where the reward increases closer to 0 as the agent comes closer to reaching the reward orbit. Once the satellite crosses within the tolerance ranges, then the reward turns positive. The partial reward is calculated by

$$r_{\text{partial}} = r_{\text{reward orbit}} \times \max(-1.0, 1 - \text{total_err}) \quad (3)$$

where $r_{\text{reward orbit}}$ is the reward associated with a particular reward orbit,

$$\text{total_err} = \text{mean}(\text{err_a}, \text{err_e}, \text{err_i})$$

and

$$\text{err_a} = \frac{|a_{\text{reward orbit}} - a_{\text{satellite}}|}{a_{\text{tolerance}}} \\ \text{err_e} = \frac{|e_{\text{reward orbit}} - e_{\text{satellite}}|}{e_{\text{tolerance}}} \\ \text{err_i} = \frac{|i_{\text{reward orbit}} - i_{\text{satellite}}|}{i_{\text{tolerance}}}$$

a , e , and i are the orbital parameters semi major axis, eccentricity, and inclination, respectively. The implementation of a partial reward makes a dense reward space out of what otherwise would have been too sparse for our agent to properly learn.

$$\mathcal{R}_{\text{minimum}} = \begin{cases} r_{\text{orbit}} & \text{if satellite in reward orbit} \\ & \text{within tolerance} \\ -500 & \text{if satellite is not alive} \\ r_{\text{reward orbit}} & \text{otherwise} \end{cases} \quad (4)$$

Where r_{orbit} is the reward value associated with the reward orbit.

F. Main Reward $\mathcal{R}$

In our main goal, our reward structure is based on three categories: proximity to the initial orbit, coming too close or colliding with RSOs, and fuel (or rather ΔV) usage.

For rewarding on orbit proximity, we created a class called OrbitProximityReward that calculates the position r and velocity v the satellite should have at the given simulation time given its initial orbit, and it compares it against the current actual r and v values to compute an overall error. The error is weighted and then subtracted from a maximum reward value that the satellite can achieve should it align with its initial orbit.

$$r_{\text{orbital proximity}} = r_{\text{max}} - (\text{pos_weight} \times \text{pos_error} \\ + \text{vel_weight} \times \text{vel_error}) \quad (5)$$

where

$$\begin{aligned} r_{max} &= 2000 \\ \text{pos_error} &= |\vec{r} - \vec{r}_{\text{target}}| \\ \text{vel_error} &= |\vec{v} - \vec{v}_{\text{target}}| \\ \text{pos_weight} &= \frac{1}{5} \\ \text{vel_weight} &= \frac{1}{15} \end{aligned}$$

and $\vec{r}_{\text{target}}$ and $\vec{v}_{\text{target}}$ are the expected position and velocity vectors of the satellite if it were to maintain its original orbit after a given time.

For RSO distance penalties, we created a class called CollisionPenalty that utilizes the relative distance between the satellite and the RSO. A penalty is given proportional to how close the satellites are to each other should they fall within the keep away distance. The full penalty value however, is awarded if the satellites collide.

$$r_{\text{collision penalty}} = \begin{cases} p & \text{if collision} \\ p \times \frac{d_{\text{keep away}} - d}{d_{\text{keep away}}} & \text{if } d < d_{\text{keep away}} \\ 0.0 & \text{otherwise} \end{cases} \quad (6)$$

where p is the penalty associated with a collision (defaulted to -1000), d is the distance between the satellite and an RSO, and $d_{\text{keep away}}$ is a designated distance for our satellite to leave between itself and the RSOs.

Finally, we defined a class called FuelPenalty that is based on BSK-RL's ResourceReward type. The class calculates a penalty reward based on the difference between the last time step's and the current time steps' ΔV available and weights it. The default weight value is 0.1.

$$r_{\text{fuel penalty}} = -\text{weight} * (\Delta V_{\text{LastStep}} - \Delta V_{\text{CurrentStep}}) \quad (7)$$

Combining these three reward functions yields our cumulative reward function.

$$\mathcal{R}_{\text{main}} = r_{\text{orbital proximity}} + r_{\text{collision penalty}} + r_{\text{fuel penalty}} \quad (8)$$

G. Observation Space $\mathcal{O}$

The observation space is defined by the combination of what we select as the observation specifications for each satellite in our environment [2]. In our minimum goal, the observation space includes only the lone satellite's body position (m) and velocity ($\frac{m}{s}$) relative to inertial origin in the planet frame and the elapsed time (normalized by simulation time limit) in the environment.

$$\mathcal{O}_{\text{minimum}} = \{r_{\text{BN_P}}, v_{\text{BN_P}}, t\} \quad (9)$$

In our main goal, our observation space is defined by the observation specifications of our main satellite in combination with those of our RSOs. Here, we include a $dv_{\text{available}}$ observation to describe the remaining fuel of the satellite. We include a simple observation specification of the number 1 for

the RSOs, to comply with BSK-RL's requirement that objects of type Satellite have an observation specification.

$$\mathcal{O}_{\text{main}} = \{r_{\text{BN_P}}, v_{\text{BN_P}}, dv_{\text{available}}, t\} \quad (10)$$

H. Observation Probability Function $\mathcal{Z}$

$\mathcal{Z}$ There is no inherent noise/uncertainty in the observations of the state unless otherwise added, so the observation probability function is a deterministic function of the state [2].

γ The discount factor we used for our minimum goal is $\gamma = 0.99$

IV. SOLUTION APPROACH

Our solution was comprised of three levels. The minimum approach was intended to increase our understanding and knowledge of BSK-RL, the main approach to create a more realistic scenario that aligned with our overall objective, and a stretch approach that sought to increase the complexity and realism of the environment.

A. Minimum

For our minimum approach, we setup a satellite using a basic dynamics model and a "MagicOrbitalManeuverFSWModel" that allowed us to utilize the ImpulsiveThrust action provided by BSK-RL. The observations given to the satellite are its position and velocity in the planet frame. This setup is shown below:

```
1 class MinimumSatellite(sats.Satellite):
2     observation_spec = [
3         obs.SatProperties(
4             dict(prop="r_BN_P", module="
5                 dynamics", norm=
6                 orbitalMotion.REQ_EARTH * 1
7                 e3),
8             dict(prop="v_BN_P", module="
9                 dynamics", norm=7616.5),
10            ),
11            obs.Time(),
12        ]
13     action_spec = [
14         act.ImpulsiveThrust(max_dv = 100.0,
15                             max_drift_duration = 5700.0)
16     ]
17     dyn_type = dyn.BasicDynamicsModel
18     fsw_type = fsw.
19     MagicOrbitalManeuverFSWModel
```

We then created a reward function as outlined in the formulation section and setup a scenario with a random "reorbit". Our assembled environment included an instance of our satellite, our reward function, and a simplistic scenario that held a reference to our "reorbit". We utilized PPO to train on the environment, and below is a snippet of the model setup:

```
1 model = PPO("MlpPolicy", # multilayer
2             perceptron policy network
3             env, learning_rate=3e-4,
4             n_steps=4096, # steps collected per
5             training update
6             batch_size=128, n_epochs=10,
7             ent_coef=0.008)
```

With the combination of our environment, observations, and PPO model, we can train the PPO algorithm to move the satellite from one orbit to another desired orbit.

Originally, we intended to implement a basic forward search for this approach before we had a deep knowledge of BSK-RL and the state space. However, due to the complexity of the environment, a forward search quickly became an infeasible approach. Instead, based on what we had found from [2], we decided to use PPO for our minimum approach. With PPO, utilized a multi-layer perceptron policy network since that is a standard and commonly implemented policy used for observations that are shaped as a flat vector. The multi-layer perceptron policy takes the state as input and outputs both values and policies [2].

B. Main

Our main approach was more complex than the minimum approach as we required more than just a single satellite. To implement this, we added a very simplistic RSO satellite that simply points nadir at all times and does not have any real observations. We also implemented a main satellite that built off of our satellite from the minimum approach. The main satellite included additional observations for available ΔV and the relative position and velocity of the RSO(s). Partial snippets of these satellite objects are as follows:

```
1 class RSOSatellite(sats.Satellite):
2     observation_spec = [
3         obs.SatProperties(dict(prop="one",
4                               fn=lambda _: 1.0)),
5     ]
6     action_spec = [act.NadirPoint(duration=1e9)]
7     dyn_type = dyn.ConjunctionDynModel
8     fsw_type = fsw.FSWModel
9
10 class MainSatellite(sats.Satellite):
11     observation_spec = [
12         obs.SatProperties(
13             ...,
14             dict(prop="dv_available"),
15         ),
16         obs.Time(),
17     ]
18     action_spec = [...]
19     observation_spec = [
20         obs.RelativeProperties(
21             dict(prop="r_DC_N", norm=1e3),
22             dict(prop="v_DC_N", norm=1e3),
23         ),
24     ]
25     dyn_type = (dyn.BasicDynamicsModel, dyn.
26                 ConjunctionDynModel)
27     fsw_type = fsw.
28                 MagicOrbitalManeuverFSWModel
```

Once again, we created reward functions based on our problem formulation described above. For our implementation, we split the reward into 3 classes. One for orbit proximity, another for checking satellite proximity and collisions, and the third for fuel (or rather ΔV) usage. Our scenario BSK-RL object for the main approach was slightly simpler than for the minimum

approach as we did not track any data within it. We also created an orbit randomizer function to place the RSOs ahead of the main satellite by some randomized amount between 120-400 km and on a collision course with the main satellite. Overall, our environment contained our satellite and RSO(s), a simplistic scenario, our reward functions, and the orbit setup function. We then utilized a technique we discovered during implementation for wrapping our environment to discretize the action and observation spaces. This wrapper essentially allowed us to extract the underlying MDP within the POMDP (which was straightforward given our observations all have 100% probability) and to discretize it for solving. We then chose DQN to train on this discretized environment. The setup for the DQN model we used is as follows:

```
1 model = DQN("MlpPolicy", # same as PPO
2             env_discrete,
3             learning_rate=3e-4, # same as PPO
4             batch_size=128, # same as PPO
5             buffer_size=20_000,
6             learning_starts=5_000, # warmup period
7             target_update_interval=5_000,
8             train_freq=4,
9             gradient_steps=1,
10            exploration_fraction=0.2,
11            exploration_final_eps=0.05,
12            policy_kwargs=dict(net_arch=[64, 64])
13 )
```

The DQN policy utilized tuned parameters from PPO as a baseline, and then we further tuned these hyperparameters throughout our training and simulation runs.

Originally, we had planned to utilize SARSA- λ for this approach, however the complexity of the state and action spaces made this algorithm not suitable for the problem, so we chose to use DQN instead. This was because SARSA- λ is an online policy, but the problem space is too complex and fast moving to fully estimate and train effectively. DQN however is an offline policy that we can train with a bunch of potential scenarios, and then let it run efficiently when we use it to predict in our problem space.

C. Stretch

For our stretch goal, we had planned to utilize DQN as our algorithm and add more complexity to the main scenario case. As previously mentioned, we utilized DQN for our main case instead of SARSA- λ , since SARSA- λ would not have worked well with the setup we had. We planned to explore additional problem complexities including adding probabilities within our thrust actions to allow a thrust action to be stronger, weaker, longer, or shorter than expected. We also wanted to add a scanning distance with which the satellite would be able to detect an RSO rather than always being aware of its location. Lastly we had wanted to add imaging of the RSOs during flybys as a task the satellite could be rewarded for. Unfortunately, we were unable to achieve these stretch goals due to a lack of time and due to the complexity of implementing satellite obstacle avoidance within the BSK-RL framework. These additional complexities would be interesting

avenues to explore in the future and would enhance our collision avoidance algorithms for more general use cases.

V. RESULTS

A. Minimum

Our minimum approach saw a fair amount of success. As Fig. 1 shows, during training, the model maintains mostly negative cumulative rewards, but does improve over time. This makes sense given the dense reward space which will give negative rewards until the satellite reaches the tolerance boundary of the desired orbit. The satellite's learning levels out for the most part after some time, but does occasionally find spots of high rewards. However, it is unable to maintain those high rewards. These training results could likely be improved in the future by implementing similar neural network and reward function optimizations to those used in [2].

Table I shows a sample of the cumulative rewards seen in Fig. 1.

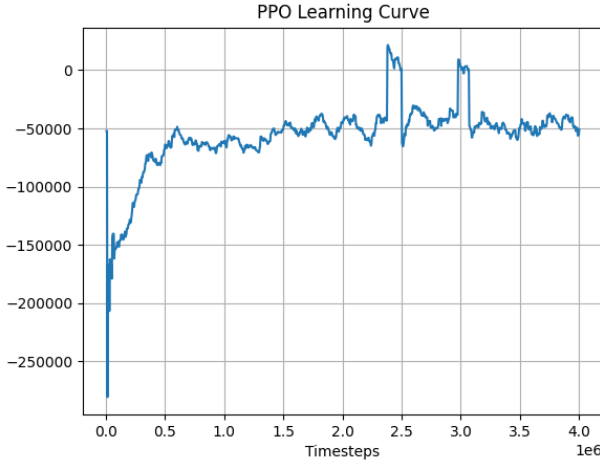


Fig. 1. PPO Learning Curve

TABLE I
PPO TRAINING DATA SAMPLE

Timestep	Episode Reward Mean
32,768	-162,298.492
98,304	-147,577.699
163,840	-143,122.080
229,376	-107,886.917
294,912	-94,585.387
360,448	-72,130.384
425,984	-79,532.520
...	...
2,383,872	21,584.891
...	...
3,825,664	-46,393.847
3,891,200	-38,122.155
3,956,736	-48,299.738

Fig. 2 shows a reward versus step graph for 1000 steps taken by the trained model on predication and evaluation. The

model is able to achieve positive increasing rewards that peak and then eventually drop back close to 0 suggesting that the satellite likely achieved the optimal orbit but then began to stray from the orbit afterwards. This may be an indication that the model was training well to achieve the optimal orbit but needs more training to maintain the orbit. Table II shows a sample of the rewards shown in the figure.

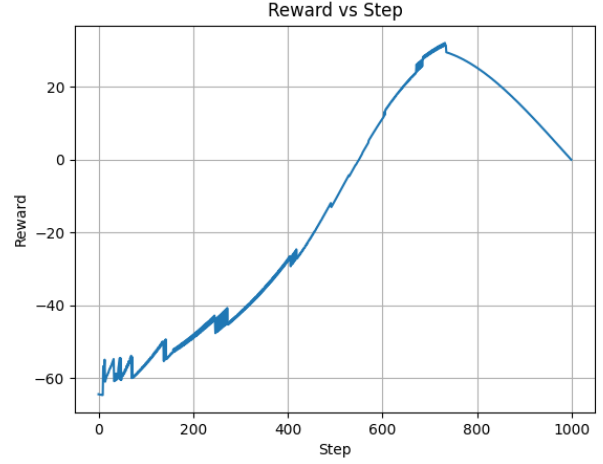


Fig. 2. Reward vs Step Plot

TABLE II
PPO TRAINED MODEL STEPS SAMPLE

Step	Reward
1	-64.360
10	-64.586
25	-57.256
50	-59.553
100	-55.939
200	-48.105
400	-28.080
600	11.039
733	32.048
800	25.326
1000	0.131

B. Main

Our main approach was moderately successful. The DQN learning curve in Fig. 3 shows a short and rapid drop followed by a rapid ascent up to a plateau. Later on, there is a brief rise and a gradual fall to the end for the mean episode reward. Unfortunately, the overall cumulative reward does remain largely negative, but considering the criteria for which it could achieve a positive reward being quite limited, this is not surprising. Further refining the weights, distances, and rewards would likely improve the overall result in the future.

Table III shows a sample of the cumulative rewards seen in Fig. 3.

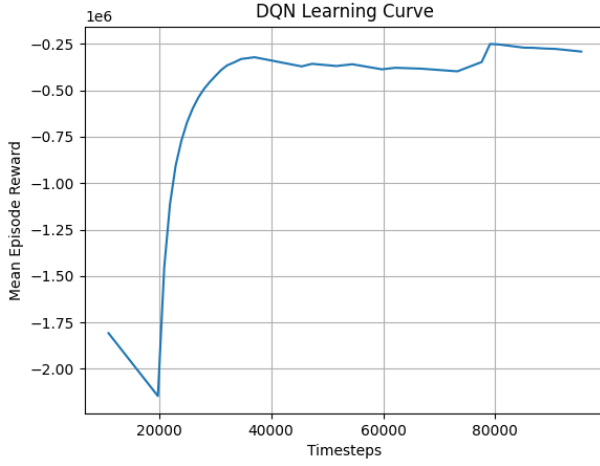


Fig. 3. DQN Learning Curve

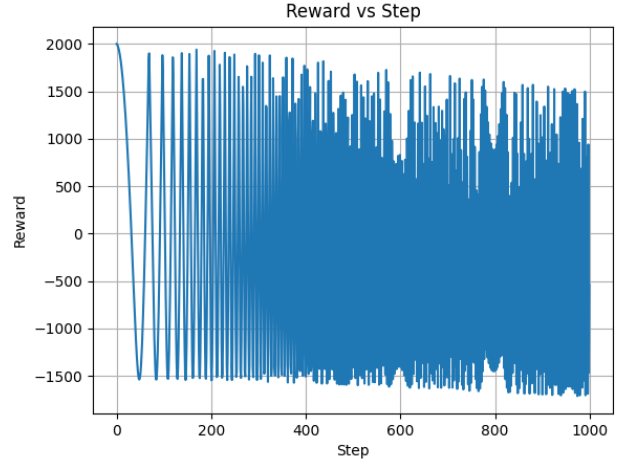


Fig. 4. DQN Reward vs Step

TABLE III
DQN TRAINING DATA SAMPLE

Timestep	Episode Reward Mean
10970	-1807814.125
19788	-2146796.097
20920	-1453556.616
21939	-1113645.023
22963	-771338.243
27037	-537301.667
30080	-421473.562
33390	-349383.191
...	
79089	-250096.227
...	
90761	-276677.388
95364	-291077.419

TABLE IV
DQN TRAINING DATA SAMPLE

Step	Reward
1	1999.693
...	
32	56.200
33	-61.726
...	
499	1250.598
500	499.296
501	-1326.680
...	
997	-1688.840
998	939.498
1000	-541.067

VI. CONCLUSION

Our minimum goal shows its value as a simple, yet related, version of our main goal. It served as good proof of concept and a useful method of familiarization for the visualization and general BSK-RL behavior with an RL algorithm. Given enough time steps, the PPO model was able to greatly improve its reward and found a policy at which its cumulative reward plateaued. Our main goal was met through our use of DQN training. Figure 3 shows successful DQN learning with possible overfitting towards the end, but it is evident that the agent learned to obtain greater and greater rewards in the complex state space.

Our results are evidence that a complex problem like autonomous satellite collision avoidance and orbit determination requires thorough formulation and rigorous testing and training. There are many undesirable circumstances that the satellite tries to avoid, like de-orbiting, collisions, and excessive fuel usage. These complications combined with the terminal nature of collisions and unintentional de-orbiting necessitate intensive training prior to deployment on spacecraft since those mistakes can be catastrophic.

Fig. 4 shows the 1000 step reward versus step graph based on the trained model. The figure shows a very cyclic behavior when running against this 1000 step case. This is likely indicative of the spacecraft moving back and forth through its orbit perhaps in an attempt to be proactive about avoiding the RSOs. The figure also shows a gradual decline in the maximum reward it achieves, although the lower bound does not change at the same rate.

Table IV shows a sample of the data shown by the figure.

In the future, we would like to explore our stretch goal to improve our models and generalize our solutions. We also gained more knowledge by working through our main goal which could be used to enhance the solution approach for our minimum goal. In particular, we would apply both PPO and DQN (as well as other suitable algorithms) to the minimum and main goals discussed in this paper and assess the varying performance. This would provide useful data for improving our algorithms and determining the optimal one for the problem. Additionally, more time to tweak our reward values would have been beneficial. Our problem was formulated such that there were multiple situations for the satellite to avoid, all of varying intensities. We would have liked to play around with the reward magnitudes for these situations to see which combination best served us for optimizing our algorithms.

VII. CONTRIBUTIONS AND RELEASE

Each of the members contributed to the project in large ways. Brian worked a great deal on the PPO usage for our minimum approach as well as modifying reward logic to make it more performant and useful. Gabriel worked largely on the code structure, setting up the scenario, satellite, and reward objects as well as some of the initial reward logic. Cannon worked on setting up visualizations for debugging, reward logic, and getting DQN running for our main approach environment. The group worked collaboratively to design the project goals and to write out each section of this document.

Due to the time constraints and complexity of setting up BSK-RL for our problem, we did not implement a full algorithm by hand, but rather focused on the implementations of our reward functions and environment setup. Instead, for our algorithms, we utilized the stable baselines implementations of PPO and DQN and tuned their parameters for our use case.

The authors grant permission for this report to be posted publicly.

ACKNOWLEDGMENT

The group gives thanks to Dr. Sunberg for a challenging, but rewarding semester of diving into the realm of decision making under uncertainty.

REFERENCES

- [1] N. Bourriez, A. Loizeau, and A. F. Abdin, "Spacecraft autonomous decision-planning for collision avoidance: A reinforcement learning approach," in *Proc. 74th Int. Astronautical Congr. (IAC)*, Baku, Azerbaijan, Oct. 2023.
- [2] L. Luna, J. Ortiz Couder, and R. A. Vargas-Acosta, "Satellite trajectory optimization via proximal policy optimization for space debris avoidance," *IEEE Access*, vol. 14, pp. 18138–18154, 2026, doi: 10.1109/ACCESS.2026.3655237.
- [3] M. A. Stephenson and H. Schaub, "BSK-RL: Modular, High-Fidelity Reinforcement Learning Environments for Spacecraft Tasking," in *Proc. 75th International Astronautical Congress (IAC)*, Milan, Italy, Oct. 14–18, 2024, Paper IAC-24-D1.2.4.
- [4] C. Mu, S. Liu, M. Lu, Z. Liu, L. Cui, and K. Wang, "Autonomous spacecraft collision avoidance with a variable number of space debris based on safe reinforcement learning," *Aerospace Science and Technology*, vol. 149, p. 109131, 2024, doi: 10.1016/j.ast.2024.109131.
- [5] C. Bombardelli and J. Hernando-Ayuso, "Optimal Impulsive Collision Avoidance in Low Earth Orbit," *Journal of Guidance, Control, and Dynamics*, vol. 38, no. 2, pp. 217–225, Feb. 2015, doi: <https://doi.org/10.2514/1.g000742>.